Convolutional Neural Networks in Computer Vision: Foundations, Advances, and Real-World Applications

A technical paper by Ng Yu Hang

P2423500

DAAA/FT/2B/23

Table of Contents

Abstract

Convolutional Neural Networks (CNNs) are the pillars of modern computer vision. This paper presents a comprehensive review of CNN fundamentals, core building blocks, and recent architectural innovations. It explains the inner workings of convolutional layers, pooling, activation functions, normalization, and classification stages. In addition, it covers advanced models such as AlexNet, VGG, ResNet, SENet, and MobileNet, highlighting their contributions and trade-offs. Beyond architecture, this paper also surveys application areas like image classification, object detection, and video prediction. To relate theory to practice, we introduce a real-time hand gesture classification system using a 1D CNN applied to MediaPipe landmark data, illustrating lightweight CNN deployment in constrained environments. Finally, we discuss current challenges and future directions in CNN research.

Introduction

Since the last decade, Convolutional Neural Networks (CNNs) have revolutionized computer vision by allowing machines to automatically extract useful and detailed features from raw images and make reliable predictions in various tasks like object recognition, scene classification as well as segmentation. CNNs are biologically very similar to receptive fields in the visual cortex of the human brain and are better than the traditional machine learning methods that relies highly on handcrafted features. CNNs are very efficient in capturing spatial hierarchies in image data using local connections and weight sharing. These features make them much more computationally efficient for larger scales of visual data. This paper first introduces the fundamental building blocks of CNNs and how they work under the hood, followed by a discussion of its major architectures and its improvements, to which then we'll cover the key application domains and conclude with a practical demonstration of such applications with my CNN – Computer Vision application for Hand Gestures, through the use of key points to train the CNN as training and validation data.

What are CNNs and How Do They Work?

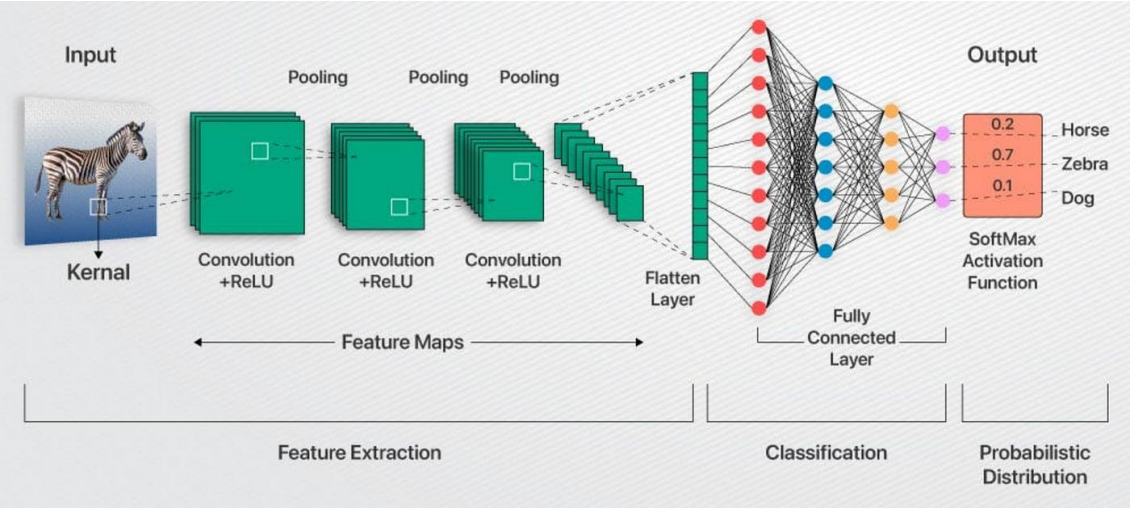

Figure 1: A diagram illustrating a Convolutional Neural Network (CNN) Architecture for Image Classification [1]

A convolutional Neural Network (CNN) is a type of Deep Learning Model that processes the data (usually images) with a grid-like topology. CNNs learn spatial hierarchies of features through its many layers of convolutions, activations, pooling etc. The input images get passed through multiple layers of processing stages, each transforming the image into a higher-level abstract representation of the image.

The first major part of any CNN is the convolutional layer, which are a set of learnable filters, also known as kernels, that slide across the input data while performing dot products of each position. Each filter is trained to extract key features of an image such as edges, patterns and textures. This results in a feature map that encodes the presence of these features spatially. Hyperparameters like stride and padding work together to determine how big or small the output will be after convolution is applied, i.e. how the filters would move and how much spatial dimensions are preserved.

Next, the image goes through the next layer: Pooling Layers. This layer would downsample the feature maps to reduce computational usage but still provide spatial invariance (The ability to recognize patterns or features regardless of which orientation the image is in). The most common form of pooling: Max Pooling, selects the highest value in a specific region of the feature map. While the other variant: Average Pooling, calculates the mean of the values instead. These layers help to abstract the image representation and reduce the possibility of overfitting.

Following the pooling layers, we have activation functions, which introduce non-linearity. This would allow the network to model complex relationships. ReLu (Rectified Linear Unit), is usually the default and most popular choice, as it outputs zero for negative values, and the identity for the positive values. This helps to reduce the problem of vanishing gradient, as well as accelerating training. Other ReLu variants such as LeakyReLu or ELU adds small gradients to help negative values learn better.

Batch Normalization is a technique that helps standardize the activations of a layer for each mini batch, which helps to stabilize and accelerate training. Another regularizer is Dropout, which randomly disables neurons during training, to prevent the model from being over-reliant on certain neurons in order to do well. They help to prevent co-adaptation as well as improve generalization.

Finally, at the end of the CNN pipeline, the fully connected layers or global pooling layers would aggregate the learned features to be used for classification. During the classification, a SoftMax activation is applied to assign a probability distribution to possible labels of the image. The training is done with backpropagation and stochastic gradient or its other variants.

Related Works and CNN Architectures

The evolution of CNN in computer vision has revolutionized over the past few years with key architectures that addresses major challenges such as network depth, computational cost, parameter size, as well as training stability, just to name a few. One of the earlier breakthroughs was AlexNet, that won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2012 by a huge margin. It pioneered innovations such as ReLu activation, dropout for regularization, as well as GPU-based parallelization. AlexNet consisted of 8 layers, including 5 Convolutional

Layers, followed by 3 fully connected layers, and reduced the top-5 classification error significantly, compared to the usual traditional methods.

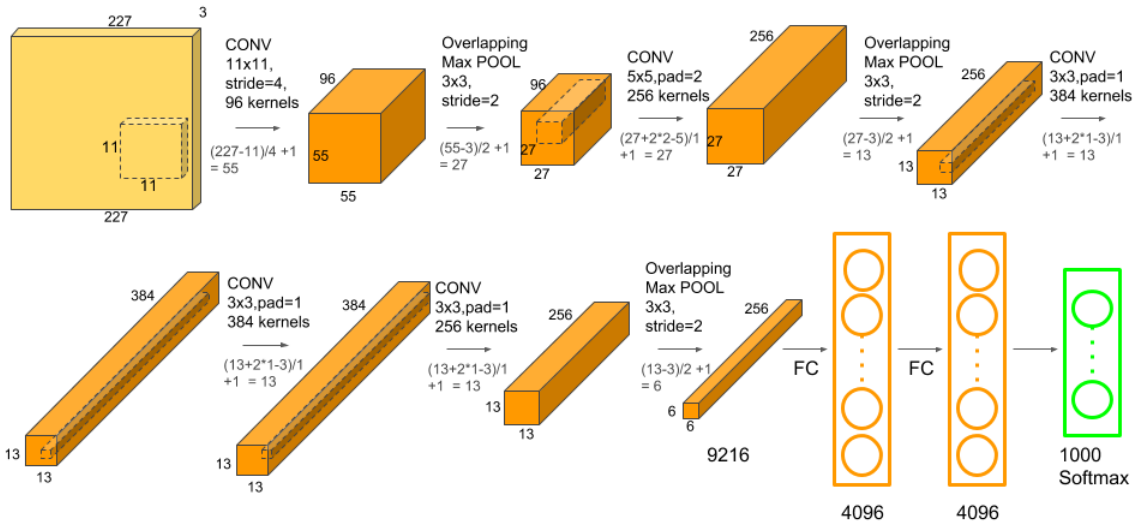

Figure 2: An Illustration of the architecture of AlexNet [2]

VGGNet, a CNN model introduced in 2014, was built on AlexNet by proposing a deeper network architecture network with smaller 3x3 filters. The VGG16 and VGG19 models successfully demonstrated that increasing depth while using simple structures can improve accuracy. That said, since VGG had a large number of parameters, it makes it extremely computationally expensive and hard to deploy on devices that have less resources.

GoogLeNet, otherwise known as Inception V1, took on the problem of computational efficiency by introducing inception modules, which used multiple convolution sizes in parallel, as well as reduce parameters through 1x1 convolutions. It was able to achieve high performance with fewer parameters than VGG. Further refinements to factorization and regularization techniques created the later versions like Inception V2 and V3, with Inception V3 being commonly used to calculate FID score due to it being trained on ResNet as well.

ResNet, proposed in 2015, revolutionized Deep learning by introducing residual connections. These skip connections help the network learn residual functions, which effectively helps to solve the degradation problem where deeper networks start performing worse. ResNet enabled architectures with over 100 layers and is still widely used today. The residual blocks allows the gradients to directly flow through shortcut connections, which helps to stabilize training.

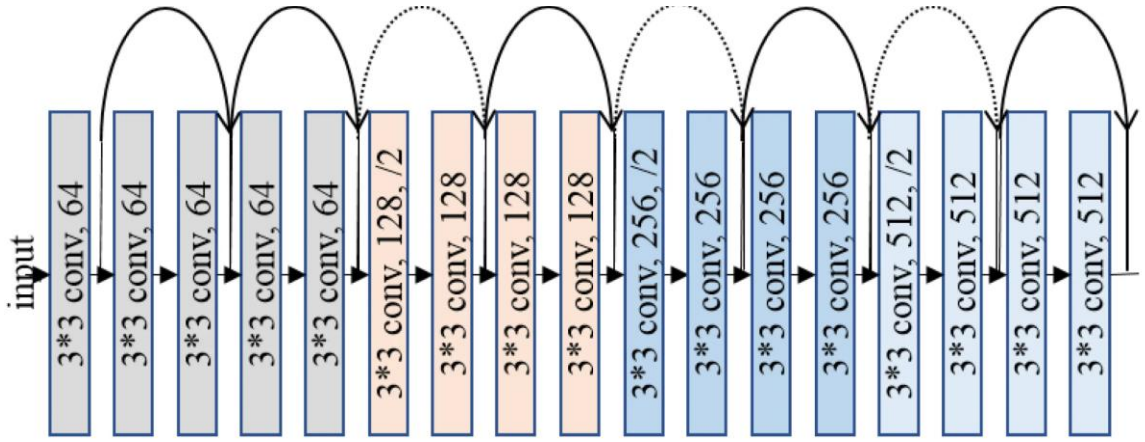

Figure 3: An Illustration of the architecture of ResNet [3]

Squeeze-and-Excitation Networks (SENet), introduced an attention mechanism at the cbannel level that allows the network to adaptively recalibrate feature maps. This mechanism significantly helped improve performance with minimal computational overhead. SE blocks were integrated into ResNet, forming SE-ResNet, which went on to achieve state-of-the-art results on ImageNet.

MobileNet, along with its successors had a strong focus on efficiency, introducing depthwise separable convolutions to reduce the number of parameters and computations. This helped allow CNNs to be deployed on mobile devices and embedded systems. One of its successors, MobileNetV2, added inverted residuals and linear bottlenecks in order to preserve representational power with a reduced cost.

Each of these above architectures challenged the limits of accuracy, speed, as well as deployment readiness. Research continues into combing CNNs with transformers, neural architecture search (NAS), as well as hybrid models to further expand their capabilities across a wider range of computer vision related tasks.

Discussion – CNNs in Computer Vision[4]

In image classification, CNNs have long been able to demonstrate their exceptional capability to recognize the patterns and assign images to their predefined categories. Models such as Alexnet and ResNet have been able to achieve great performances in benchmarks like the ImageNet Large Scale Visual Recognition Challenge (ILSVRC), where they surpassed traditional methods by a huge margin. These very advancements have enabled machines to reliably distinguish thousands of objects with high accuracies.

Beyond classification, object detection represents another major use case. Unlike classification, detection involves both identifying what objects as well as where they're located. This is commonly accompanied by using region proposal frameworks such as R-CNN, YOLO and

SSD. These CNN-based detectors are widespread now with uses in domains such as autonomous vehicles, where real-time object localization is extremely important for navigation and safety.

Image segmentation further extends CNN's capabilities by classifying each pixel in an image. Architectures such as U-Net and Mask R-CNN allow in depth scene understanding, which is very useful in medical imaging for tasks like tumor boundary delineation or organ segmentation.

In real world, Convolutional Neural Networks (CNNs) is used in many important things:

- **Autonomous driving:** CNNs assist vehicles in recognizing their surroundings. It is able to recognize other vehicles, pedestrians, and signs. The car's modules are able to make well-informed decisions thanks to this information.
- **Medical imaging:** CNNs are also used to verify MRI, CT, and X-ray scans. It can spot minor issues that medical professionals might miss. This helps detect stroke or cancer early.
- **Augmented reality:** CNNs enable the placement of digital things in the actual world. To ensure that virtual items remain in their proper locations, it tracks forms and objects' locations.

Furthermore, **pretrained CNN models** like ResNet50, InceptionV3, and EfficientNet provide a solid foundation for the creation of new applications. These models can be optimized for particular tasks through the use of transfer learning, especially in situations where labeled data is difficult to locate. Because previously learned visual elements are reused, this method significantly cuts down on training time and frequently results in improved performance. However, despite their strength, CNNs come with a few challenges:

- Particularly for highly specialized domains, they frequently need for huge, labelled datasets, which are extremely expensive to create and time-consuming to prepare, making it even more difficult to have large, high-quality datasets easily accessible.
- CNNs frequently operate as "black boxes," devoid of interpretability; they might be extremely sensitive to changes in minor factors like lighting, orientation, or occlusion, making them less adaptive without appropriate augmentation. This affects its dependability when applied in crucial fields like law enforcement or healthcare.
- If not properly examined and managed, CNNs may potentially inherit biases from training data arising from incorrect labelling or class imbalances, leading to social or demographic imbalances.

Looking ahead, there are still several intriguing research avenues being investigated. Models can learn visual representations from unlabelled data through self-supervised learning. Graph convolutional networks and attention mechanisms can enhance interpretability and spatial reasoning. Furthermore, integrating CNNs with generative models or reinforcement learning may enable more sophisticated vision systems with the ability to reason, plan, or imagine.

In conclusion, CNNs have changed the field of computer vision and made it possible for more significant real-world uses. Their ongoing development is probably going to pave the way for future developments in all AI-powered perceptual tasks, not just vision.

A practical example of CNN usage in Computer Vision

Through earlier discussions, we have established the usefulness and potential of the usage of CNNs in Computer Vision, with many being able to improve efficiency and accuracy for various professions. Here I have built an example of such an application which uses a CNN to identify hand gestures in real time. Applications like such are similar to those used in traffic law enforcements, where accurate car tracking is required. Through further refinements, applications like this could be used to identify incidents, such as an AI that identifies whether someone is drowning in a pool or whether there's a fight happening.

I will be providing a brief walk-through about how my application works, starting first with how my application acquires a dataset:

```python
# === Config ===
output_csv = 'gesture_data.csv'
label_map = {
    ord('1'): 'fist',
    ord('2'): 'palm',
    ord('3'): 'peace',
    ord('4'): 'thumb',
    ord('5'): 'three'

}
# === Setup ===
mp_hands = mp.solutions.hands
hands = mp_hands.Hands(static_image_mode=False, max_num_hands=1,
                       min_detection_confidence=0.7, min_tracking_confidence=0.5)
cap = cv2.VideoCapture(0)

current_label = None
data_buffer = []
print("[INFO] Press 1-5 to set label, S to save sample, Q to quit.")

while True:
    ret, frame = cap.read()
    if not ret:
        continue

    frame = cv2.flip(frame, 1)
    image_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = hands.process(image_rgb)

    if results.multi_hand_landmarks:
        for hand_landmarks in results.multi_hand_landmarks:
            # Draw Landmarks
            mp.solutions.drawing_utils.draw_landmarks(
                frame, hand_landmarks, mp_hands.HAND_CONNECTIONS)

            # Get 63 Landmark values (x, y, z)
            flat_keypoints = []
            for lm in hand_landmarks.landmark:
                flat_keypoints.extend([lm.x, lm.y, lm.z])

            # Display current label
            if current_label:
                cv2.putText(frame, f"Label: {current_label}", (10, 40),
                            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

            # Save on 's' press
            key = cv2.waitKey(1)
            if key == ord('s') and current_label:
                data_buffer.append(flat_keypoints + [current_label])
                print(f"[SAVED] {current_label} ({len(data_buffer)} samples)")
            elif key in label_map:
                current_label = label_map[key]
                print(f"[LABEL] Switched to '{current_label}'")
```

```
            elif key == ord('q'):
                break
    else:
        key = cv2.waitKey(1)
        if key in label_map:
            current_label = label_map[key]
            print(f"[LABEL] Switched to '{current_label}'")

        elif key == ord('q'):
            break

    cv2.imshow("Data Collector", frame)

# === Save CSV ===
if data_buffer:
    with open(output_csv, 'w', newline='') as f:
        writer = csv.writer(f)
        for row in data_buffer:
            writer.writerow(row)
    print(f"[DONE] Saved {len(data_buffer)} samples to {output_csv}")
else:
    print("[WARN] No samples collected.")

cap.release()
cv2.destroyAllWindows()
```

Figure 4: Code that allows user to gather keypoints of certain hand poses and assign it a label for CNN training purposes.

```
# === Load and preprocess ===
df = pd.read_csv('gesture_data.csv', header=None)
X = df.iloc[:, :-1].values.astype(np.float32)
y = df.iloc[:, -1].values
X = X.reshape(-1, 21, 3)

le = LabelEncoder()
y_encoded = le.fit_transform(y)
y_cat = utils.to_categorical(y_encoded)

with open('label_map.json', 'w') as f:
    json.dump({int(k): v for k, v in zip(le.transform(le.classes_), le.classes_)}, f)

X_train, X_test, y_train, y_test = train_test_split(
    X, y_cat, test_size=0.2, random_state=42, stratify=y_cat
)

# === Model ===
model = models.Sequential([
    layers.Conv1D(64, kernel_size=3, activation='relu', input_shape=(21, 3)),
    layers.BatchNormalization(),
    layers.Conv1D(128, kernel_size=3, activation='relu'),
    layers.GlobalMaxPooling1D(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.4),
    layers.Dense(y_cat.shape[1], activation='softmax')
])

model.compile(
    optimizer=Adam(learning_rate=0.0005),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# === Callbacks ===
es = callbacks.EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
rlr = callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3, verbose=1)

# === Train ===
history = model.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=50,
    batch_size=32,
    shuffle=True,
    callbacks=[es, rlr]
)

# === Save ===
model.save('gesture_cnn_model.h5')
print("[DONE] Tuned CNN model trained and saved as 'gesture_cnn_model.h5'")
```

Figure 5: Code that uses the keypoints to train a 1D CNN model of 3 values with 2 Convolutional Layers – total of 7 layers

As shown above, the dataset is created by running the first cell of the notebook, which would first open a webcam. By pressing different keys to label the keypoints getting saved, the model receives much more realistic environments that it will be tested on. The keypoints are saved in a CSV file, which are then fed to a 1D CNN model with the following configuration:

2 Conv1D layers → BatchNorm → GlobalMaxPooling1D → Dense(64) → Dropout(0.4) → Output Dense(softmax)

This model's weights are then stored as a .h5 file and is now ready to run. To do so, just run the below cell after training the model (run the cell above) once:

```
# === Load model and label map ===
model = load_model('gesture_cnn_model.h5')
with open('label_map.json', 'r') as f:
    label_map = json.load(f)
    label_map = {int(k): v for k, v in label_map.items()}

# === Setup MediaPipe ===
mp_hands = mp.solutions.hands
hands = mp_hands.Hands(static_image_mode=False, max_num_hands=1,
                       min_detection_confidence=0.7, min_tracking_confidence=0.5)
mp_draw = mp.solutions.drawing_utils

# === Webcam ===
cap = cv2.VideoCapture(0)

# === Initialize FPS timing ===
prev_time = 0

while True:
    ret, frame = cap.read()
    if not ret:
        continue

    # FPS timing start
    curr_time = time.time()
    fps = 1 / (curr_time - prev_time)
    prev_time = curr_time

    frame = cv2.flip(frame, 1)
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    result = hands.process(rgb)

    if result.multi_hand_landmarks:
        for hand_landmarks in result.multi_hand_landmarks:
            mp_draw.draw_landmarks(frame, hand_landmarks, mp_hands.HAND_CONNECTIONS)

            # Extract and reshape 21 keypoints (63 values → (21, 3))
            points = []
            for lm in hand_landmarks.landmark:
                points.append([lm.x, lm.y, lm.z])
            keypoints = np.array(points).reshape(1, 21, 3)

            # Predict
            prediction = model.predict(keypoints, verbose=0)
            class_id = int(np.argmax(prediction))
            gesture = label_map[class_id]
            confidence = float(np.max(prediction))

            # Display gesture
            cv2.putText(frame, f"{gesture} ({confidence:.2f})", (10, 40),
                        cv2.FONT_HERSHEY_SIMPLEX, 1.2, (0, 255, 0), 3)

    # Display FPS
    cv2.putText(frame, f"FPS: {fps:.2f}", (10, 80),
                cv2.FONT_HERSHEY_SIMPLEX, 1.0, (255, 0, 0), 2)

    cv2.imshow("Gesture Prediction", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Figure 6: Running the trained CNN model with computer vision (using mediapipe) to detect which handpose is being shown.

Figure 7: A preview of how the model works

Conclusion:

CNN has made significant progress throughout many years, going from just pure image classification, to getting implemented into applications that are being used in our daily lives, with the most common example being the facial recognition function in our phones. They too use CNN to identify our facial features. This is a great example of our progress in the usage of CNNs in Computer Vision.

As more methods get discovered on how to optimize speed, accuracy and efficiency, we can expect to see much better and quicker models in the coming years.

References:

[1] Ravjot Singh (2021) *Decoding CNNs: A Beginner's Guide to Convolutional Neural Networks and Their Applications*. Available at: https://ravjot03.medium.com/decoding-cnns-a-beginners-guide-to-convolutional-neural-networks-and-their-applications-1a8806cbf536 (Accessed: 31 July 2025).

[2] Musstafa (2023) *AlexNet In-Depth*. Available at: https://musstafa0804.medium.com/alexnet-in-depth-92a5d6b67df1 (Accessed: 31 July 2025).

[3] Insightful Tscript (n.d.) *ResNet*. Available at: https://insightfultscript.com/collections/programming/neural-network/resnet/ (Accessed: 31 July 2025).

[4] AI and Insights (2024) *Convolutional Neural Networks (CNNs) in Computer Vision*. Available at: https://medium.com/@AIandInsights/convolutional-neural-networks-cnns-in-computer-vision-10573d0f5b00 (Accessed: 31 July 2025).